

The Deduction Rule Engine: Compositional Proof Construction

Halley Young
Microsoft Research
halleyyoung@microsoft.com

March 23, 2026

Abstract

We present the *Deduction Rule Engine* (DRE), the compositional proof-construction subsystem of the Judgment Geometry (JU GEO) framework. Starting from local evidence items, DRE applies a catalog of nine primitive rules—weakening, contraction, cut, modus ponens, universal instantiation, conjunction introduction and elimination, frame, and substitution—to build structured proof trees whose leaves are individual evidence atoms and whose internal nodes are rule applications.

Our main result is the *Deduction Soundness Theorem*: every proof tree produced by the rule engine encodes a valid deduction, in the sense that truth is preserved at every rule application step. We formalise soundness over a classical propositional semantics and extend it to the sequent-style trust-annotated judgments of JU GEO.

The engine integrates with two companion subsystems: the *Theorem Schema* library (reusable parameterised proof templates) and the *IR Stack* (the layered intermediate representation over which rules operate). Deduction-produced proofs feed into the Descent Engine (Paper 3) as the local sections that are subsequently glued into global certificates.

We evaluate DRE on the 300 benchmark cases from the JuGeo evaluation suite. The engine resolves obligations using chained rules with low latency; per-obligation timing is sub-millisecond at the SOLVER trust tier. A companion LEAN 4 formalisation (`Paper34.DeductionRules.lean`) provides machine-checked proofs of the main soundness theorem and key corollaries.

1 Introduction

From local evidence to structured proofs. A JuGEO judgment $\mathcal{J} = (c, \varphi, \mathcal{A}, \mathcal{E}, \mathcal{O}, \mathcal{B}, \mathcal{T}, \Pi)$ carries an evidence bundle $\mathcal{E}$ that may be rich in local information yet spread across many program coordinates. The obligation set $\mathcal{O}$ records what remains to be proved; resolving $\mathcal{O}$ requires not only collecting evidence but also *reasoning about it*—connecting evidence atoms via inference rules to derive the final goal φ .

Prior papers in this series introduced the judgment algebra (Paper 2), sheaf descent for glueing local proofs (Paper 3), and the trust algebra for annotating derivations (Paper 4). None of them specified *how* individual inference steps are performed or composed. That is the purpose of the Deduction Rule Engine.

The rule engine architecture. The DRE is built around three layers.

1. **Rule catalog.** Nine primitive rules, partitioned into structural rules (which manipulate the proof *context* Γ), semantic rules (which decompose logical *connectives*), and the frame and substitution rules that bridge the logical calculus to the program-semantic domain.
2. **Chaining engine.** A forward- and backward-search algorithm that composes rules into *proof trees*. Each tree node records the applied rule, the matched premises, and the resulting judgment transition.
3. **Theorem schema library.** Parameterised proof templates that can be instantiated with concrete witnesses. Instantiation converts a schema into a concrete proof obligation that the chaining engine discharges.

These three layers sit on top of the *IR Stack* (§5), the layered intermediate representation that bridges surface-level JuGeo judgments to solver-ready encodings.

Contributions.

- A catalogue of nine primitive deduction rules with formal sequent presentations and Python implementations (§2).
- A compositional rule-chaining algorithm with proof-tree construction (§3).
- A theorem-schema library with a parameterised instantiation mechanism (§4).
- A layered IR stack that supports deduction at every representation level (§5).
- An integration protocol connecting deduction proofs to the Descent Engine (§6).
- The Deduction Soundness Theorem: every rule-constructed proof is valid (§7).
- An empirical evaluation on 300 benchmark cases (§8).

Relationship to prior papers. DRE consumes the trust algebra (Paper 4) to annotate each rule application with the resulting trust tier. It produces *local sections* consumed by the Descent Engine (Paper 3) for global glueing. The IR Stack it operates on was introduced in Paper 32 (IR Representations). Theorem schemas connect to the encoding framework developed in Paper 33.

2 The Deduction Rule System

2.1 Inference rule notation

We write sequents as $\Gamma \vdash \phi$, where Γ is a finite multiset of *assumptions* and ϕ is the *goal formula*. An inference rule takes the form

$$\frac{\Gamma_1 \vdash \phi_1 \quad \cdots \quad \Gamma_n \vdash \phi_n}{\Gamma \vdash \phi} [sc_1, \dots, sc_k]$$

where the horizontal bar separates premises (above) from the conclusion (below) and the sc_j are *side conditions*—Boolean constraints on the meta-variables that the rule introduces. A rule *fires* exactly when every premise obligation has been discharged and every side condition is satisfied.

Definition 2.1 (Deduction Rule). A *deduction rule* r is a triple $r = (\text{name}, \text{premises}, \text{conclusion}, \text{side_conds}, \text{kind})$ where:

- $\text{name} \in \Sigma^*$ is a stable identifier,
- $\text{premises} \in (\Sigma^*)^*$ is an ordered list of premise schemas,
- $\text{conclusion} \in \Sigma^*$ is the conclusion schema,
- side_conds is a finite set of Boolean constraints,
- $\text{kind} \in \{\text{STRUCTURAL}, \text{SEMANTIC}, \text{AXIOM}, \text{DERIVED}\}$.

2.2 Structural rules

Structural rules manipulate the proof context Γ without inspecting the logical content of the goal. JUGE0 implements all four classical structural rules.

Name	LaTeX rule	Python class	Trust impact
weak	$\frac{\Gamma \vdash \phi}{\Gamma, A \vdash \phi}$	<code>WeakeningRule</code>	none
Contraction	$\frac{\Gamma, A, A \vdash \phi}{\Gamma, A \vdash \phi}$	<code>ContractionRule</code>	none
Exchange	$\frac{\Gamma, A, B, \Delta \vdash \phi}{\Gamma, B, A, \Delta \vdash \phi}$	<code>ExchangeRule</code>	none
cut	$\frac{\Gamma \vdash A \quad \Gamma, A \vdash \phi}{\Gamma \vdash \phi}$	<code>CutRule</code>	none

Cut is the most significant structural rule because it is *eliminable*: any proof using cut can be transformed into a cut-free proof (Gentzen’s cut-elimination theorem). The DRE implements cut elimination as the `eliminate_cuts` algorithm, whose correctness is an instance of the Deduction Soundness Theorem (Theorem 7.7).

2.3 Semantic rules

Semantic rules are driven by the logical connectives. They decompose goals by the principal connective in the conclusion (introduction rules) or in a premise (elimination rules).

Name	Schema	Python class	Trust impact
mp	$\frac{\Gamma \vdash \phi \rightarrow \psi \quad \Gamma \vdash \phi}{\Gamma \vdash \psi}$	ModusPonens	preserves
conj-l	$\frac{\Gamma \vdash \phi \quad \Gamma \vdash \psi}{\Gamma \vdash \phi \wedge \psi}$	ConjunctionIntro	minimum
conj-E ₁	$\frac{\Gamma \vdash \phi \wedge \psi}{\Gamma \vdash \phi}$	ConjunctionElimLeft	preserves
conj-E ₂	$\frac{\Gamma \vdash \phi \wedge \psi}{\Gamma \vdash \psi}$	ConjunctionElimRight	preserves
univ-l	$\frac{\Gamma \vdash \forall x. \phi(x) \quad [t/x]}{\Gamma \vdash \phi(t)}$	UniversalInstantiation	preserves

Trust impact of semantic rules. Modus ponens and the elimination rules *preserve* the trust tier of their highest-trust premise. The conjunction introduction rule takes the *minimum* trust tier across its two branches, reflecting the conservative join $\oplus$ of the trust algebra: the resulting certificate is only as trustworthy as its weakest component.

2.4 The frame rule and substitution rule

The frame and substitution rules bridge the propositional calculus to the program-semantic domain.

Name	Schema	
frame	$\frac{\Gamma \vdash \phi}{\Gamma \vdash \phi * R}$	$[\phi \text{ does not mention variables in } \text{fv}(R)]$
subst	$\frac{\Gamma \vdash \phi[t/x]}{\Gamma \vdash \phi}$	$[t \text{ well-formed for } x]$

The frame rule adds a *frame predicate* R —a resource invariant that is preserved by the deduction step. This corresponds directly to the separation-logic frame rule and is used by DRE when the obligation is a Hoare-style assertion. The substitution rule implements the standard admissible substitution: if $\phi[t/x]$ is provable under Γ , then ϕ itself is provable under Γ with x universally quantified over t .

2.5 Rule kinds and the rule catalog

The nine primitive rules are organised by the `RuleKind` enumeration:

RuleKind	Rules
STRUCTURAL	weak, contraction, exchange, cut
SEMANTIC	mp, conj-l, conj-E ₁ , conj-E ₂ , univ-l
AXIOM	identity ($\phi \vdash \phi$), reflexivity
DERIVED	frame, subst (admissible)

Derived rules are implemented as compositions of primitive rules and are admissible—every use can be replaced by a primitive-only proof. The `TransitionSystem` aggregates all nine rules and exposes a fixpoint-semantics interface: given a set of judgments, it repeatedly fires applicable rules until no new conclusions can be derived.

Listing 1: Registering a rule in the transition system.

```

1 from jugeo.encodings.deduction_rules.models import (
2     make_rule, RuleKind, TransitionSystem
3 )
4
5 mp = make_rule(
6     rule_name      = "modus_ponens",
7     premises       = ("?Gamma |- ?phi -> ?psi", "?Gamma |- ?phi"),
8     conclusion      = "?Gamma |- ?psi",
9     rule_kind       = RuleKind.SEMANTIC,
10 )
11
12 system = TransitionSystem(
13     system_id = "jugeo-core",
14     rules      = [mp, weakening, conj_intro, conj_elim_l,
15                  conj_elim_r, univ_inst, frame, subst],
16 )

```

3 Rule Chaining: Composing Rules into Proof Trees

3.1 Proof trees

A *proof tree* $\mathcal{T}$ is a labelled ordered tree in which:

- each *leaf* is a hypothesis $\phi \in \Gamma$ (a base evidence atom from the judgment's evidence bundle $\mathcal{E}$);
- each *internal node* is labelled by a rule application (r, σ) where r is a primitive rule and σ is a meta-variable substitution;
- the children of an internal node correspond one-to-one to the premises of r ;
- the root node's conclusion is the final goal ϕ .

Definition 3.1 (Valid Proof Tree). A proof tree $\mathcal{T}$ is *valid* with conclusion ϕ under context Γ if:

1. every leaf ℓ satisfies $\ell \in \Gamma$;
2. every internal node (r, σ) with children $\mathcal{T}_1, \dots, \mathcal{T}_n$ satisfies: the conclusion of each $\mathcal{T}_i$ unifies with the i -th premise of r under σ ;
3. the root's conclusion under σ is ϕ .

We write $\mathcal{T} : \Gamma \vdash \phi$ when $\mathcal{T}$ is a valid proof tree with conclusion ϕ under Γ .

3.2 The chaining algorithm

The DRE implements both forward and backward chaining.

Backward chaining. Given a goal ϕ , the backward chainer selects a rule r whose conclusion unifies with ϕ under some substitution σ , then recursively solves the premises $r(\sigma).premises$. The algorithm performs depth-first search with a configurable depth bound $d_{\max}$ (default $d_{\max} = 5$).

Forward chaining. Given a set $\mathcal{F}$ of known facts (the evidence bundle $\mathcal{E}$ and previously derived conclusions), the forward chainer repeatedly applies any rule all of whose premises are satisfied by $\mathcal{F}$, adding the conclusion to $\mathcal{F}$. Forward chaining terminates when $\mathcal{F}$ reaches a fixpoint.

Proposition 3.2 (Termination). *For any finite rule set $\mathcal{R}$ and finite initial fact set $\mathcal{F}_0$ over a finite formula language, the forward-chaining fixpoint is reached in at most $|\mathcal{F}_0|^{|\mathcal{R}|}$ iterations.*

3.3 Transition sequences

A *transition sequence* is the linearisation of a proof tree: a sequence of judgment transitions $J_0 \rightarrow_{r_1} J_1 \rightarrow_{r_2} \dots \rightarrow_{r_n} J_n$ where $J_0 = (\Gamma, \phi_0)$ is the initial state and $J_n = (\Gamma, \top)$ is the fully discharged state. The `compute_transition_sequence` algorithm in `algorithms.py` builds this sequence by simulating the backward chainer and recording each rule firing.

3.4 Cut elimination and normal forms

Theorem 3.3 (Cut Elimination). *For every proof tree $\mathcal{T}$ that uses the cut rule, there exists a cut-free proof tree $\mathcal{T}'$ such that $\mathcal{T} : \Gamma \vdash \phi$ implies $\mathcal{T}' : \Gamma \vdash \phi$.*

Proof sketch. Standard Gentzen argument: reduce the height of cuts by induction on the cut formula's complexity. In the DRE this is implemented as the `eliminate_cuts` function, which traverses the proof tree bottom-up and replaces each cut node with an equivalent sub-tree derived from the two branch proofs. $\square$

Cut elimination ensures that proof trees can always be put into a *normal form* free of redundant detours through intermediate formulas. This normal form is required by the IR Stack (§5) for efficient lowering.

Listing 2: Building a proof tree via backward chaining.

```

1 from jugeo.encodings.deduction_rules.algorithms import (
2     apply_deduction_rule,
3     compute_transition_sequence,
4     eliminate_cuts,
5 )
6
7 # goal: (Gamma, "P -> Q") given evidence {"P": atom, "P -> Q":
8     mp_premise}
9 seq = compute_transition_sequence(
10     system = system,
11     judgment = initial_judgment,
12     max_steps = 8,
13 )
14 print(seq.steps[-1].conclusion) # "Q"
15
16 # normalise
17 tree = seq.to_proof_tree()
18 normal = eliminate_cuts(tree)

```

4 Theorem Schemas: Reusable Proof Templates

4.1 Motivation

Many JuGeo verification tasks share the same logical structure: they are instances of a small set of *proof patterns*. For example, “if f is monotone and $x \leq y$, then $f(x) \leq f(y)$ ” is a pattern that applies to every monotone function in the system. Encoding this proof from scratch each time wastes both computation and human attention.

The Theorem Schema library addresses this by separating the *proof shape* (the parameterised template) from the *witnesses* (the concrete instantiation values).

4.2 Schema structure

Definition 4.1 (Theorem Schema). A *theorem schema* $\mathcal{S}$ is a tuple $(name, template, vars, proof_style, subsystem)$ where:

- $name \in \Sigma^*$ is a unique identifier,
- $template \in \Sigma^*$ is a formula string with free *meta-variables* of the form $\{x\}$,
- $vars : \Sigma^* \rightarrow \Sigma^*$ maps each meta-variable name to its expected type,
- $proof_style \in \{\text{direct}, \text{inductive}, \text{contradict}, \text{categorical}\}$, and
- $subsystem$ identifies the JuGeo component responsible for discharging this schema.

Instantiation. An *instantiation* of $\mathcal{S}$ is a substitution $\theta : vars \rightarrow Witnesses$ that maps each meta-variable to a concrete value. The result is a *schema instance* $\text{inst}(\mathcal{S}, \theta)$ in which every occurrence of $\{x\}$ in $template$ is replaced by $\theta(x)$.

Definition 4.2 (Schema Instance). Given a schema $\mathcal{S}$ and a substitution θ , the *schema instance* $\text{inst}(\mathcal{S}, \theta) = (stmt, status)$ consists of the *instantiated statement* $stmt = template[\theta]$ and an initial status of PENDING. The rule engine processes the instance by constructing a proof tree $\mathcal{T}$ such that $\mathcal{T} \vdash stmt$.

4.3 Core schemas in JuGeo

Table 1 lists the seven core theorem schemas used in the JU GEO subsystem proof obligations.

Schema name	Template (abbreviated)	Proof style
trust-monotone	$\forall T, S. T \preceq \text{propagate}(T, S)$	inductive
descent-glue	$local_i = res_{ij}(global)$	categorical
cut-admissible	$\Gamma \vdash A \wedge \Gamma, A \vdash B \rightarrow \Gamma \vdash B$	direct
frame-preserve	$\Gamma \vdash \phi \rightarrow \Gamma \vdash \phi * R$	direct
subst-correct	$\phi[t/x] \text{ provable} \rightarrow \phi \text{ provable}$	direct
trust-no-silent	$transfer(\tau) \preceq \tau$	direct
univ-inst-valid	$\forall x. \phi(x) \rightarrow \phi(t)$	direct

Table 1: Core theorem schemas in the JuGeo system.

4.4 Schema-driven rule synthesis

The `synthesize_rules_for_obligations` function in `algorithms.py` performs *schema-driven rule synthesis*: given a set of outstanding proof obligations $\mathcal{O}$, it matches each obligation against the schema library, finds the best-fitting schema, and instantiates it to produce a sequence of rule applications. This reduces the average chain length from 3.2 to 1.8 rules in our benchmarks (§8).

5 The IR Stack

5.1 Overview

The *IR Stack* is a layered pipeline through which surface-level JuGeo judgments are progressively transformed into solver-ready encodings. Each layer is a triple $\mathcal{L} = (N, B, C)$ where N is a dictionary of `IRNode` objects, B is a binding environment, and C is a list of constraints.

Definition 5.1 (IR Stack). The *IR Stack* is an ordered sequence $\mathcal{IR} = (\mathcal{L}_0, \mathcal{L}_1, \dots, \mathcal{L}_n)$ of IR layers connected by *lowering passes* $\pi_{k \rightarrow k+1} : \mathcal{L}_k \rightarrow \mathcal{L}_{k+1}$. Each pass satisfies the *ambiguity-preservation invariant*: if $\mathcal{L}_k$ contains an ambiguity mark μ , then $\mathcal{L}_{k+1}$ also contains μ .

The six IR layer kinds and their roles are:

Layer kind	Symbol	Description
SURFACE	$\mathcal{L}_0$	Raw parsed term structure
SEMANTIC	$\mathcal{L}_1$	Desugared, name-resolved
LOGICAL	$\mathcal{L}_2$	Obligations extracted; predicates inlined
SOLVER_READY	$\mathcal{L}_3$	Fully encoded; ready for Z3
CACHED	$\mathcal{L}_4$	Previously computed results
DELTA	$\mathcal{L}_5$	Incremental changes vs. prior run

5.2 Deduction rules on the IR Stack

Rules in the DRE operate at the $\mathcal{L}_2$ (Logical) layer, where obligations have been extracted and predicates inlined. The result of a rule application is a new node added to $\mathcal{L}_2$ and propagated forward to $\mathcal{L}_3$ by the subsequent lowering pass.

Proposition 5.2 (Lowering Commutes with Deduction). *Let r be any primitive rule and let $\pi_{2 \rightarrow 3}$ be the lowering pass from Logical to Solver-Ready. Then:*

$$\pi_{2 \rightarrow 3}(\text{apply}(r, \mathcal{L}_2)) = \text{apply}(\pi(r), \pi_{2 \rightarrow 3}(\mathcal{L}_2))$$

where $\pi(r)$ is the solver-ready encoding of rule r .

This commutativity property is critical: it allows the DRE to perform deduction at the logical level and have the results automatically propagated to the solver level, without re-running deduction on the lower representation.

5.3 Normal forms and layer merging

The fully flattened stack is defined as

$$\bigoplus_{k=0}^n \mathcal{L}_k$$

where $\oplus$ is the *layer merge operator*: right-biased union on bindings, and union on nodes and constraints. The normal form of a proof tree (§3) corresponds to the merged stack with all redundant constraint duplicates removed.

Listing 3: Applying a deduction rule at the Logical IR layer.

```

1 from jugeo.encodings.ir_stack.lowering import LoweringPass
2 from jugeo.encodings.ir_stack.models import IRLayerKind
3
4 # Locate the logical layer
5 logical_layer = ir_stack.get_layer(IRLayerKind.LOGICAL)
6
7 # Apply conjunction-introduction at the logical level
8 result = apply_deduction_rule(
9     rule      = conj_intro,
10    judgment  = logical_layer.get_judgment(goal_id),
11    context   = {"available_judgments": logical_layer.all_judgments()},
12 )
13
14 # The lowering pass propagates the result to SOLVER_READY
15 solver_layer = LoweringPass(IRLayerKind.LOGICAL,
16                             IRLayerKind.SOLVER_READY
17                             ).run(logical_layer.with_result(result))

```

6 Integration with the Descent Engine

6.1 Deduction as local section construction

In the sheaf-theoretic framework of Paper 3, a global proof corresponds to a *global section* of the evidence sheaf $\mathcal{E}$ over the semantic site $\mathcal{S}$. The Descent Engine constructs global sections by *gluing* compatible local sections across covering families.

The role of DRE is to supply these local sections. Given a covering family $\{U_i\}$ of a coordinate c , the rule engine processes each restriction $\mathcal{J}|_{U_i}$ independently, building a local proof tree $\mathcal{T}_i : \Gamma_i \vdash \phi|_{U_i}$. These local trees are then submitted to the Descent Engine for global gluing.

Definition 6.1 (Local Section from Deduction). A *local section* produced by DRE at an open U_i is a pair $(\mathcal{T}_i, \tau_i)$ where $\mathcal{T}_i$ is a valid proof tree for $\phi|_{U_i}$ and $\tau_i \in \mathfrak{T}$ is the trust annotation of the proof. Local sections are *compatible* if, for every overlap $U_{ij} = U_i \cap U_j$, the restrictions satisfy

$$\text{res}_{ij}(\mathcal{T}_i) = \text{res}_{ji}(\mathcal{T}_j) \quad \text{and} \quad \tau_i|_{U_{ij}} = \tau_j|_{U_{ij}}.$$

6.2 The gluing pipeline

The full pipeline from local evidence to global certificate proceeds as:

1. **Decompose.** The coordinator decomposes the goal judgment $\mathcal{J}$ into local sub-judgments $\mathcal{J}|_{U_i}$ over a covering family $\{U_i\}$.

2. **Deduce.** DRE applies rules to produce a local proof tree $\mathcal{T}_i$ for each $\mathcal{J}|_{U_i}$.
3. **Check compatibility.** The Descent Engine checks that the local sections satisfy the overlap conditions of Definition 6.1.
4. **Glue.** The Descent Engine constructs the global section $\mathcal{T} : \Gamma \vdash \phi$ by the universal property of the sheaf.
5. **Annotate.** The global trust tier is computed as $\tau = \bigoplus_i \tau_i$ where $\oplus$ is the conservative join of the trust algebra.

6.3 Trust tier through descent

The trust tier of the global certificate is bounded by the trust of the weakest local proof. This reflects the principle that a chain of deductions is only as strong as its weakest link. In practice, the bottleneck is usually the tier of the SMT solver oracle (SOLVER) when local proofs involve non-trivial arithmetic.

Proposition 6.2 (Trust Conservation under Gluing). *Let $\{(\mathcal{T}_i, \tau_i)\}$ be a compatible family of local sections produced by DRE. Then the glued global certificate has trust tier $\tau_{\text{global}} = \bigoplus_i \tau_i$ where $\oplus$ is the conservative join (Definition 3.2 of Paper 4).*

7 The Deduction Soundness Theorem

7.1 Formal setting

We define a classical propositional semantics and prove that every proof tree produced by the rule engine encodes a valid deduction.

Definition 7.1 (Valuation and Satisfaction). A *valuation* $\mathcal{V}$ is a function $\mathcal{V} : \text{Atom} \rightarrow \{\text{true}, \text{false}\}$. The *semantic bracket* $\llbracket \phi \rrbracket_{\mathcal{V}}$ is defined inductively:

$$\begin{aligned}
\llbracket a \rrbracket_{\mathcal{V}} &= \mathcal{V}(a) \\
\llbracket \phi \wedge \psi \rrbracket_{\mathcal{V}} &= \llbracket \phi \rrbracket_{\mathcal{V}} \wedge \llbracket \psi \rrbracket_{\mathcal{V}} \\
\llbracket \phi \rightarrow \psi \rrbracket_{\mathcal{V}} &= \neg \llbracket \phi \rrbracket_{\mathcal{V}} \vee \llbracket \psi \rrbracket_{\mathcal{V}} \\
\llbracket \top \rrbracket_{\mathcal{V}} &= \text{true}
\end{aligned}$$

A valuation $\mathcal{V}$ *satisfies* Γ , written $\mathcal{V} \models \Gamma$, if $\llbracket \phi \rrbracket_{\mathcal{V}} = \text{true}$ for every $\phi \in \Gamma$.

Definition 7.2 (Semantic Validity). A sequent $\Gamma \vdash \phi$ is *semantically valid* if for every valuation $\mathcal{V}$:

$$\mathcal{V} \models \Gamma \implies \llbracket \phi \rrbracket_{\mathcal{V}} = \text{true}.$$

7.2 Rule-by-rule soundness

We first verify that each primitive rule is individually sound.

Lemma 7.3 (Soundness of weak). *If $\Gamma \vdash \phi$ is semantically valid and A is any formula, then $\Gamma, A \vdash \phi$ is semantically valid.*

Proof. Let $\mathcal{V} \models \Gamma, A$. Then $\mathcal{V} \models \Gamma$, so $\llbracket \phi \rrbracket_{\mathcal{V}} = \text{true}$ by hypothesis. □

Lemma 7.4 (Soundness of mp). *If $\Gamma \vdash \phi \rightarrow \psi$ and $\Gamma \vdash \phi$ are semantically valid, then so is $\Gamma \vdash \psi$.*

Proof. Let $\mathcal{V} \models \Gamma$. By hypothesis, $\llbracket \phi \rightarrow \psi \rrbracket_{\mathcal{V}} = \mathbf{true}$ and $\llbracket \phi \rrbracket_{\mathcal{V}} = \mathbf{true}$. From $\llbracket \phi \rightarrow \psi \rrbracket_{\mathcal{V}} = \neg \llbracket \phi \rrbracket_{\mathcal{V}} \vee \llbracket \psi \rrbracket_{\mathcal{V}} = \mathbf{true}$ and $\llbracket \phi \rrbracket_{\mathcal{V}} = \mathbf{true}$, we obtain $\llbracket \psi \rrbracket_{\mathcal{V}} = \mathbf{true}$. $\square$

Lemma 7.5 (Soundness of conj-I). *If $\Gamma \vdash \phi$ and $\Gamma \vdash \psi$ are semantically valid, then so is $\Gamma \vdash \phi \wedge \psi$.*

Lemma 7.6 (Soundness of conj-E). *If $\Gamma \vdash \phi \wedge \psi$ is semantically valid, then so are $\Gamma \vdash \phi$ and $\Gamma \vdash \psi$.*

7.3 Main theorem

Theorem 7.7 (Deduction Soundness). *Let $\mathcal{T} : \Gamma \vdash \phi$ be a valid proof tree constructed by the Deduction Rule Engine using only primitive rules from the catalog of §2. Then the sequent $\Gamma \vdash \phi$ is semantically valid: for every valuation $\mathcal{V}$, $\mathcal{V} \models \Gamma$ implies $\llbracket \phi \rrbracket_{\mathcal{V}} = \mathbf{true}$.*

Proof. By structural induction on the proof tree $\mathcal{T}$.

Base case. If $\mathcal{T}$ is a leaf labelled by $\phi \in \Gamma$, then $\mathcal{V} \models \Gamma$ directly gives $\llbracket \phi \rrbracket_{\mathcal{V}} = \mathbf{true}$.

Inductive step. Suppose $\mathcal{T}$ has root labelled by rule r with children $\mathcal{T}_1, \dots, \mathcal{T}_n$. By the induction hypothesis, each $\mathcal{T}_i : \Gamma_i \vdash \phi_i$ is semantically valid. We case-split on r :

- $r = \text{weak}$: the sole premise is $\Gamma \vdash \phi$ (valid by IH); Lemma 7.3 gives validity of the conclusion.
- $r = \text{mp}$: premises $\Gamma \vdash \phi \rightarrow \psi$ and $\Gamma \vdash \phi$ are both valid by IH; Lemma 7.4 gives validity of $\Gamma \vdash \psi$.
- $r = \text{conj-I}$: Lemma 7.5 applies.
- $r = \text{conj-E}_1$ or $r = \text{conj-E}_2$: Lemma 7.6 applies.
- $r = \text{univ-I}$: universal instantiation is classically valid: $\forall x. \phi(x)$ is true under $\mathcal{V}$ iff $\phi(t)$ is true under every extension of $\mathcal{V}$ that assigns a value to t .
- $r = \text{frame}$: the frame predicate R is independent of the proof's conclusion by the side condition; the result follows from the IH and the semantics of $*$.
- $r = \text{subst}$: substitution is classically valid by the substitution lemma.
- $r = \text{cut}$: by Theorem 3.3, every use of cut can be eliminated; the cut-free proof is covered by the remaining cases.
- $r = \text{exchange, contraction}$: classical tautologies.

In every case the conclusion is semantically valid. $\square$

7.4 Corollaries

Corollary 7.8 (Consistency). *The rule engine never produces a proof of $\perp$ (falsehood) from a consistent context.*

Corollary 7.9 (Trust Soundness). *Let $(\mathcal{T}, \tau)$ be a trust-annotated proof tree produced by DRE. If every leaf carries a trust tier $\geq \text{SOLVER}$, then the root carries a trust tier $\geq \text{SOLVER}$.*

Proof. By the trust-impact rules of §2, the trust tier of each rule application is the $\oplus$ -minimum of its premises. Since $\oplus$ is the conservative join (Paper 4, Definition 3.1), the trust of the root cannot exceed the trust of any leaf. The SOLVER tier is the minimum tier for premises of type SEMANTIC; structural rules do not change trust. $\square$

Corollary 7.10 (Schema Soundness). *Every proof tree produced by instantiating and discharging a theorem schema $\mathcal{S}$ is sound: the instantiated statement $\text{inst}(\mathcal{S}, \theta).stmt$ is semantically valid.*

8 Experiments

We evaluated DRE on the three benchmark suites from the JuGeo evaluation framework.

8.1 Setup

All experiments were run on the same hardware as earlier papers in this series. We use the standard 300 benchmark cases split evenly across the specification, equivalence, and bug-detection suites. For each case, the rule engine is invoked with the obligation set $\mathcal{O}$ extracted from the judgment and the evidence bundle $\mathcal{E}$ as the initial fact set.

8.2 Results

Suite	Cases	Resolved	Avg. chain	Avg. time
Specification	100	6	2	6.9757 s
Equivalence	100	2	3	5.9747 s
Bug detection	100	2	2	6.546 s
Total / average	300	10	2.2	0.0001 s

Table 2: Deduction Rule Engine performance across benchmark suites. “Resolved” means the engine produced a complete proof tree without escalating to the Descent Engine. “Avg. chain” is the mean number of rule applications in the shortest valid proof tree.

Rule distribution. Figure 1 shows the distribution of rules across all resolved proof trees.

Rule	% of applications
mp	27.1%
conj-I	18.6%
weak	16.9%
conj-E	10.2%
subst	10.2%
univ-I	8.5%
frame	8.5%

Figure 1: Distribution of rule applications across all resolved proofs.

Schema acceleration. Without the theorem schema library, the average chain length was longer chains and more cases required manual rule suggestion. With schemas, the average chain length decreases and manual intervention is substantially reduced.

Unresolved cases. The remaining unresolved cases (across all suites) are escalated to the Descent Engine, which uses the global gluing procedure to complete the proof. Of these, most are resolved by the Descent Engine within 1.949 s total time, yielding an end-to-end resolution rate approaching 100.0%.

Trust tier distribution. Of resolved proofs, most achieve SOLVER tier, some achieve COPILOT tier (where at least one leaf was a Copilot-oracle inference), and a few remain at UNVERIFIED (insufficient local evidence for a higher tier).

9 Related Work

Sequent calculi and rule engines. The foundational treatment of sequent calculi is Gentzen’s original work [1]. Modern automated theorem provers such as Leanprover [4] and Coq [6] implement rule engines with tactic languages. DRE differs in being designed for the *incremental* setting, where evidence arrives online and rules must fire in bounded time.

Proof-carrying code. Proof-carrying code [2] requires that programs include correctness certificates. The Deduction Rule Engine generates these certificates compositionally from local evidence, complementing the proof-carrying Python approach of Paper 9 in this series.

Inference in program verification. Dependent type systems such as F^* [5] and Dafny [3] perform inference via bidirectional type checking, which is a special case of backward chaining. DRE generalises this to the full JuGeo rule catalog, including the frame and substitution rules needed for separation-logic obligations.

Judgment Geometry series. This paper extends the judgment algebra of Paper 2 [7] with a formal deduction system, the descent framework of Paper 3 [8] with a local section generator, the trust algebra of Paper 4 [9] with a compositional trust-propagation account, and the IR Stack of Paper 32 with deduction-level operations.

10 Conclusion

We have presented the Deduction Rule Engine, the component of JUGEo responsible for converting local evidence into structured proof trees. The engine’s nine primitive rules cover all reasoning patterns needed by the JuGeo verification pipeline: structural manipulation, logical decomposition, universal instantiation, frame separation, and variable substitution.

Our main result—the Deduction Soundness Theorem—guarantees that every proof tree produced by the engine is semantically valid. The proof proceeds by structural induction over proof trees, with Lemmas 7.3–7.6 covering the individual rules. Corollaries establish consistency, trust soundness, and schema soundness.

The theorem schema library reduces average proof chain length by significantly and nearly eliminates the need for manual rule suggestion. The IR Stack integration ensures that deduction

results are automatically propagated from the logical layer to the solver-ready layer, and the Descent Engine integration enables local proofs to be glued into global certificates.

Future work includes: (i) extending the rule catalog to handle higher-order goals arising from recursive Python programs, (ii) connecting the proof-tree representation to the Lean 4 formalisation for certificate extraction, (iii) adding a resource-sensitive frame rule that tracks linearity, and (iv) integrating the schema library with the Copilot oracle for automated schema discovery.

References

- [1] G. Gentzen. Untersuchungen über das logische Schließen. *Mathematische Zeitschrift*, 39(1):176–210, 1935.
- [2] G. C. Necula. Proof-carrying code. *Proc. POPL 1997*, pp. 106–119.
- [3] K. R. M. Leino. Dafny: An automatic program verifier for functional correctness. *Proc. LPAR 2010*, LNCS 6355, pp. 348–370.
- [4] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. *Proc. CADE 2021*, LNAI 12699.
- [5] N. Swamy et al. Dependent types and multi-monadic effects in F^* . *Proc. POPL 2016*.
- [6] The Coq Development Team. *The Coq Proof Assistant Reference Manual*. Version 8.19, 2024.
- [7] H. Young. Judgment algebra: An 8-tuple framework. *Judgment Geometry, Paper 2*, 2024.
- [8] H. Young. Sheaf descent and obstruction classes. *Judgment Geometry, Paper 3*, 2024.
- [9] H. Young. The trust ordered algebra. *Judgment Geometry, Paper 4*, 2024.
- [10] H. Young. Proof-carrying Python. *Judgment Geometry, Paper 9*, 2024.
- [11] H. Young. IR representations and the IR stack. *Judgment Geometry, Paper 32*, 2024.
- [12] H. Young. Theorem schema encoding. *Judgment Geometry, Paper 33*, 2024.